



TẬP ĐOÀN CÔNG NGHIỆP – VIỄN THÔNG QUÂN ĐỘI

BÁO CÁO MINI - PROJECT

Đề tài: Database Quality Library

Học viên

Họ và tên Trịnh Quang Lâm

Ngày sinh 14/01/2004

Số điện thoại 0971624914

Email trinquanglam2k4@gmail.com

Chương trình Viettel Digital Talent 2026

Lĩnh vực: Software Engineer

Mentor: Nguyễn Văn Báu

Đơn vị: Tổng Công ty Dịch vụ Tài chính số Viettel – VDS

Link sản phẩm: <https://github.com/quanglam04/database-quality-library/>

Hà Nội, 07/2026

Lời mở đầu

Trong quá trình phát triển phần mềm doanh nghiệp, chất lượng tương tác với cơ sở dữ liệu thường là yếu tố quyết định hiệu năng hệ thống trên môi trường production. Các vấn đề như thiếu chỉ mục, truy vấn N+1, hay dùng **SELECT *** thường không lộ diện trong giai đoạn phát triển do dữ liệu còn ít, nhưng gây ra sự cố nghiêm trọng khi hệ thống vận hành thực tế.

Báo cáo này trình bày quá trình nghiên cứu, thiết kế và triển khai thư viện **Database Quality Library** — một thư viện Java chuyên dụng nhằm lấp đầy khoảng trống giữa các công cụ monitoring lớp ứng dụng hiện có và nhu cầu phân tích chéo thực tế giữa cấu trúc schema và hành vi SQL tại runtime.

Đề tài được thực hiện trong khuôn khổ chương trình **Viettel Digital Talent 2026**, lĩnh vực Software Engineer, dưới sự hướng dẫn của mentor tại đơn vị Tổng Công ty Dịch vụ Tài chính số - VDS.

Tác giả trân trọng cảm ơn mentor và các thành viên trong chương trình đã hỗ trợ, góp ý trong suốt quá trình thực hiện đề tài.

Mục lục

Lời mở đầu	1
1 Giới thiệu	6
1.1 Đặt vấn đề	6
1.1.1 Nỗi đau 1: Production chậm nhưng dev không tái hiện được	6
1.1.2 Nỗi đau 2: Gặp N+1 nhưng không biết dòng code nào gây ra	6
1.1.3 Nỗi đau 3: Schema và code không đồng bộ	6
1.1.4 Nỗi đau 4: Phát hiện vấn đề quá muộn	6
1.1.5 Nỗi đau 5: Phải dùng nhiều tool rời rạc	7
1.2 Tư duy giải quyết	7
1.2.1 Nguyên tắc 1: Hoàn toàn không xâm lấn code nghiệp vụ	7
1.2.2 Nguyên tắc 2: Phân tích chéo DDL và SQL runtime	7
1.2.3 Nguyên tắc 3: Chỉ rõ vị trí lỗi trong code nghiệp vụ	8
1.2.4 Nguyên tắc 4: Phát hiện sớm, gợi ý hành động cụ thể	8
1.3 Mục tiêu	8
1.4 Phạm vi	9
2 Nội dung và phương pháp	9
2.1 Kiến trúc tổng thể	9
2.1.1 Tách Collection Layer và Analysis Layer	9
2.1.2 Các thành phần chính	10
2.1.3 Luồng dữ liệu	10
2.2 Cơ chế hoạt động	11
2.2.1 Thu thập metadata (DDL)	11
2.2.2 Thu thập metrics (runtime)	11
2.2.3 Phân tích Execution Plan với cache	12
2.3 Mô tả Rule Engine	13
2.3.1 Cấu trúc Rule Engine	13
2.3.2 Nhóm Rule dựa trên DDL	13
2.3.3 Nhóm Rule dựa trên Metrics	13
2.3.4 Nhóm Rule dựa trên EXPLAIN	14
2.3.5 Tính minh bạch đầu vào	15
2.3.6 Scoring	15
2.4 Dashboard và Metrics API	15
2.4.1 Dashboard Realtime	15
2.4.2 Metrics API và các Endpoint	16
2.5 Hướng dẫn tích hợp	16
2.5.1 Yêu cầu môi trường	16
2.5.2 Thêm dependency	17
2.5.3 Cấu hình	17
2.5.4 Spring Boot zero-config	18
3 Kết quả thực hiện và đánh giá	19
3.1 Môi trường thử nghiệm	19
3.2 Giá trị đem lại	19
3.2.1 Pain point 1: Production chậm nhưng không tái hiện được	19
3.2.2 Pain point 2: Gặp N+1 nhưng không biết ở đâu	19

3.2.3	Pain point 3: Schema và code không đồng bộ	20
3.2.4	Pain point 4: Phát hiện vấn đề quá muộn	20
3.2.5	Pain point 5: Nhiều tool rời rạc	20
3.3	Kết quả tổng hợp	20
3.4	Đánh giá overhead khi tích hợp thư viện	20
3.4.1	Overhead trên hot path (mỗi câu SQL)	21
3.4.2	Overhead trên Analysis Layer (background)	21
3.4.3	Overhead bộ nhớ	21
3.4.4	Đo overhead thực tế bằng Apache Bench	22
3.4.5	Khuyến nghị triển khai	23
3.5	Dashboard demo	24
3.6	Đánh giá tổng thể	28
4	Kết luận	29
4.1	Tóm tắt kết quả	29
4.2	Hướng phát triển tương lai	29
	Tài liệu tham khảo	30

Danh sách hình vẽ

1	Tab Overview — score badge ở header (kèm tooltip công thức tính), nút Run Analysis Now, 4 metric cards, latency percentiles, top tables và top slow queries kèm nút EXPLAIN	24
2	Tab Findings — danh sách findings sắp xếp theo severity, mỗi finding có rule name, severity badge, message với metrics thực tế, recommendation và vị trí <code>calledFrom</code> trong code nghiệp vụ	24
3	Tab Findings — filter theo severity để tập trung xử lý vấn đề ưu tiên cao trước	25
4	Tab Collected Queries — input transparency của rule engine. Hiển thị toàn bộ SQL pattern đã được normalize, kèm call count, avg/min/max duration và vị trí code gọi nhiều nhất	25
5	Tab Schema Snapshot — input transparency của rule engine cho DDL. Hiển thị cấu trúc schema thu thập qua <code>DatabaseMetaData</code> : tables, columns kèm kiểu dữ liệu và ràng buộc nullable, indexes, foreign keys	26
6	Modal Execution Plan — hiển thị khi bấm nút EXPLAIN trên slow query. Bao gồm findings phát hiện được từ plan (full scan, low selectivity, sort to disk) và raw JSON/XML output của EXPLAIN	26
7	Tab AI (phần trên) — AI-ready context: prompt structured được build từ schema summary, top findings, metrics, slow queries. Có thể copy hoặc export để dùng với bất kỳ LLM nào	27
8	Tab AI (phần dưới) — AI Insights được render từ markdown response của LLM, hỗ trợ code block, table, list. LLM phân tích sâu hơn và đề xuất SQL fix cụ thể	27
9	Tab Project — thông tin runtime: database product/version, framework, ORM, connection pool, JVM heap usage với progress bar, và uptime	28

Danh sách bảng

1	Các package chính của thư viện	10
2	Hỗ trợ EXPLAIN theo vendor	12
3	Nhóm Rule phân tích Schema	13
4	Nhóm Rule phân tích Metrics	14
5	Nhóm Rule dựa trên EXPLAIN	14

1 Giới thiệu

1.1 Đặt vấn đề

Sự cố hiệu năng liên quan đến database là một trong những vấn đề khó chịu và tốn kém nhất trong vòng đời một sản phẩm phần mềm. Khác với lỗi logic — thường được bắt ngay khi test — các vấn đề về database thường lặng lẽ tích tụ trong giai đoạn phát triển và chỉ bùng phát khi hệ thống chạy thật với dữ liệu lớn và lưu lượng cao. Khi đó, việc tìm ra nguyên nhân gốc rễ là một quá trình mệt mỏi, đầy phỏng đoán. Phần này mô tả năm tình huống điển hình mà gần như developer nào cũng đã từng gặp.

1.1.1 Nỗi đau 1: Production chậm nhưng dev không tái hiện được

Vào một buổi sáng, team support báo: “Trang danh sách đơn hàng load mất 8 giây trên production.” Developer mở môi trường dev, click cùng chức năng đó — load trong 200ms. Mở staging — 350ms. Không một dấu hiệu nào cho thấy có vấn đề.

Nguyên nhân thường là database dev chỉ có vài chục bản ghi, trong khi production có hàng trăm nghìn. Một câu query thiếu index có thể chạy nhanh với 50 rows nhưng chậm gấp ngàn lần với 500.000 rows. Phần lớn các công cụ APM (Application Performance Monitoring) chỉ báo “query này chậm” mà không trả lời được *tại sao* — không cho biết câu query đó đang full table scan, đang dùng nhầm index, hay đang join sai cách.

1.1.2 Nỗi đau 2: Gặp N+1 nhưng không biết dòng code nào gây ra

Hibernate Statistics ghi nhận: “Trong 2 phút vừa qua, có 1.247 query `SELECT * FROM employees WHERE id = ?`.” Developer biết đây là N+1 pattern điển hình của lazy loading — nhưng câu hỏi tiếp theo mới khó: *dòng code nào gây ra?*

Trong một codebase Spring Boot trung bình, có hàng chục service method gọi đến `Employee`. Phần lớn các tool chỉ cho biết **có** N+1 mà không chỉ **ở đâu**. Developer phải mở từng repository, từng service, đọc lại logic, đặt breakpoint — một quá trình tốn nhiều giờ chỉ để tìm ra một dòng code thiếu `@EntityGraph`.

1.1.3 Nỗi đau 3: Schema và code không đồng bộ

Bảng `orders` có một index trên cột `status` — tạo từ ba năm trước cho một báo cáo đã không còn dùng. Index này vẫn ngốn dung lượng và làm chậm mọi lệnh `INSERT/UPDATE`, nhưng không ai dám xóa vì *không biết còn dùng không*.

Ngược lại, cột `email` của `customers` được dùng trong `WHERE` của hàng chục query nhưng vẫn chưa có index. Vấn đề này chỉ lộ ra khi performance team kiểm tra ngẫu nhiên, hoặc khi production gặp sự cố.

Schema và hành vi truy vấn ở runtime tồn tại như hai thực thể tách rời — một bên là metadata tĩnh, một bên là pattern động. Việc đối chiếu thủ công gần như bất khả thi với các hệ thống có hàng trăm bảng.

1.1.4 Nỗi đau 4: Phát hiện vấn đề quá muộn

Một anti-pattern điển hình: cột `salary` được khai báo `FLOAT` thay vì `DECIMAL`, gây sai số tích lũy khi tính tổng lương. Hoặc cột `birth_date` khai báo `VARCHAR` thay vì `DATE`, khiến mọi query so sánh ngày bị sai.

Những vấn đề này hiển nhiên trong DDL — bất kỳ ai review schema đều thấy — nhưng vòng đời thực tế thường bỏ qua bước review này. Đến khi phát hiện thì:

- Schema đã được deploy lên production
- Dữ liệu sai đã tích lũy
- Migration đòi hỏi downtime hoặc backfill phức tạp

1.1.5 Nỗi đau 5: Phải dùng nhiều tool rời rạc

Để chẩn đoán một vấn đề database, developer phải mở:

- Hibernate Statistics (xem query count)
- p6spy (xem SQL được sinh ra)
- DBeaver / phpMyAdmin (chạy EXPLAIN thủ công)
- Grafana / APM (xem latency)
- Schema tool riêng (xem DDL)

Mỗi tool cung cấp một mảnh ghép, không tool nào tổng hợp được toàn cảnh. Quá trình chuyển đổi giữa các tool tốn thời gian và mất context.

1.2 Tư duy giải quyết

Thay vì xây dựng một tool monitoring tổng quát chỉ làm tốt hơn các tool hiện có một chút, đề tài tiếp cận vấn đề từ góc nhìn của developer: *thiếu cái gì và tại sao các tool hiện tại không lấp được khoảng trống đó*. Từ năm nỗi đau trên, đề tài rút ra bốn nguyên tắc thiết kế cốt lõi.

1.2.1 Nguyên tắc 1: Hoàn toàn không xâm lấn code nghiệp vụ

Developer không nên phải thêm annotation, kế thừa class trừu tượng, hay sửa repository để dùng được tool. Điều này dẫn đến lựa chọn kiến trúc **intercept ở lớp JDBC** — nơi mà mọi ORM (Hibernate, MyBatis, jOOQ) và mọi framework (Spring Boot, Quarkus, Micronaut) đều phải đi qua.

Cụ thể, thư viện bao bọc `DataSource` gốc bằng `QualityDataSource` theo Decorator Pattern. Mọi câu SQL chạy qua đều được intercept, đo thời gian, capture vị trí gọi — mà không một dòng code nghiệp vụ nào phải thay đổi. Spring Boot user thậm chí không cần viết một dòng config nào nhờ auto-configuration.

1.2.2 Nguyên tắc 2: Phân tích chéo DDL và SQL runtime

Các tool hiện có thường chỉ làm tốt *một mặt* — hoặc phân tích query (p6spy), hoặc phân tích schema (DBeaver). Giá trị thực sự đến từ việc **đối chiếu hai nguồn dữ liệu này với nhau**:

- Cột `x` được dùng trong WHERE của 30 query → nhưng schema không có index cho cột này → đề xuất tạo index.

- Index `idx_y` có trong schema → nhưng EXPLAIN của tất cả query không dùng đến → cảnh báo index có thể thừa.
- Bảng `z` có cột FK `user_id` → nhưng không có index trên cột này → JOIN sẽ full scan.

Đây chính là khoảng trống mà thư viện lấp vào — không thay thế các tool hiện có, mà bổ sung góc nhìn cross-reference mà chúng không có.

1.2.3 Nguyên tắc 3: Chỉ rõ vị trí lỗi trong code nghiệp vụ

Việc phát hiện ra một N+1 pattern là chưa đủ — developer cần biết *ở đâu trong code* để fix. Thư viện capture stack trace mỗi lần SQL được thực thi, sau đó lọc bỏ các frame thuộc framework (Spring, Hibernate, JDBC driver), proxy class (Hibernate proxy, CGLIB, JDK dynamic proxy), và chính thư viện. Frame còn lại là vị trí code nghiệp vụ thật.

Kết quả là mỗi finding kèm theo trường `calledFrom` dạng: `io.demo.hr.service.impl.LeaveServiceImpl:85 -> toResponse()` — developer mở đúng file, đúng dòng, fix trong vài phút thay vì vài giờ.

1.2.4 Nguyên tắc 4: Phát hiện sớm, gợi ý hành động cụ thể

Tool không chỉ nói “có vấn đề” — mà phải nói **vấn đề gì, ở đâu, và làm sao fix**. Mỗi finding trả về có 4 trường:

- **rule** — loại vấn đề
- **severity** — mức độ nghiêm trọng (CRITICAL/HIGH/MEDIUM/WARNING)
- **message** — mô tả cụ thể kèm metrics thực tế (chạy bao nhiêu lần, tổng thời gian, max/min/avg)
- **recommendation** — gợi ý fix cụ thể (CREATE INDEX..., Dùng JOIN FETCH..., Đổi sang DECIMAL(19,4)...)

Thư viện cũng phát hiện vấn đề DDL ngay khi app khởi động qua `DatabaseMetaData` — không cần chờ traffic, không cần chờ đến khi vào production.

1.3 Mục tiêu

Từ bốn nguyên tắc trên, đề tài đặt mục tiêu xây dựng một thư viện Java có khả năng:

- **Thu thập trong suốt** — wrap `DataSource` theo Decorator Pattern để intercept toàn bộ SQL ở lớp JDBC, không yêu cầu thay đổi code nghiệp vụ. Thu thập DDL qua `DatabaseMetaData` chuẩn JDBC.
- **Phân tích chéo** — triển khai tối thiểu 10 rules đối chiếu DDL và SQL runtime, mỗi finding có severity, mô tả vấn đề, khuyến nghị khắc phục và vị trí code.
- **Minh bạch input** — không chỉ hiển thị findings, mà còn hiển thị *đầu vào* của rule engine (SQL pattern + metrics + DDL snapshot) để user có thể verify kết quả phân tích.

- **Tách collection và analysis** — collection chạy realtime với overhead minimal, analysis chạy theo interval cấu hình được, đảm bảo không ảnh hưởng performance của request thật.
- **Cấu hình linh hoạt** — cho phép user tự điều chỉnh interval analysis, threshold các rule, bật/tắt từng tính năng qua `application.properties`.
- **Output đa dạng và độc lập** — embedded dashboard, JSON report, AI-ready context. Toàn bộ output hoạt động đầy đủ mà không cần cấu hình AI.

1.4 Phạm vi

Trong phạm vi:

- Java 17+, Maven/Gradle, mọi cơ sở dữ liệu quan hệ có hỗ trợ JDBC Driver
- Tích hợp với bất kỳ ứng dụng Java nào sử dụng JDBC, không phụ thuộc framework
- Deployment dưới dạng thư viện JAR thuần (không có server riêng)

Ngoài phạm vi:

- Phân tích reactive/R2DBC (chỉ hỗ trợ JDBC đồng bộ)
- Lưu trữ lịch sử phân tích giữa các lần restart (in-memory only)
- Hỗ trợ NoSQL

2 Nội dung và phương pháp

2.1 Kiến trúc tổng thể

2.1.1 Tách Collection Layer và Analysis Layer

Một quyết định kiến trúc quan trọng của thư viện là **tách rõ hai layer chạy song song**:

- **Collection Layer (realtime)** — chạy mỗi khi SQL được thực thi. Layer này phải cực nhẹ: chỉ ghi metric vào in-memory map, không chạy rule, không gọi EXPLAIN. Mục tiêu: overhead < 1ms mỗi query.
- **Analysis Layer (theo interval)** — chạy ở background thread theo chu kỳ cấu hình được (mặc định 5 phút, có thể đặt 24h). Layer này thực hiện các tác vụ nặng: warm-up ExplainCache, chạy rule engine, lưu kết quả vào `AnalysisResultStore`.

Việc tách layer mang lại các lợi ích:

1. **Hot path nhẹ** — request thật của user không bị chậm vì rule engine
2. **API response nhanh** — dashboard đọc cache, không trigger phân tích mới
3. **Tránh race condition** — chỉ một thread chạy phân tích
4. **Phân biệt rõ realtime metrics và scheduled findings** — user thấy SQL count tăng ngay lập tức, nhưng findings chỉ refresh theo interval

Người dùng có thể trigger phân tích ngay bằng nút **Run Analysis Now** trên dashboard mà không cần đợi đến interval tiếp theo — hữu ích khi đang test fix một issue.

2.1.2 Các thành phần chính

Thư viện được tổ chức thành các package độc lập, mỗi package chịu trách nhiệm một giai đoạn trong pipeline:

Bảng 1: Các package chính của thư viện

Package	Mô tả
<code>com.dbquality.core</code>	JDBC Interception Layer. Bộ 4 lớp Decorator: <code>QualityDataSource</code> , <code>QualityConnection</code> , <code>QualityPreparedStatement</code> , <code>QualityStatement</code> .
<code>com.dbquality.collector</code>	Collection Layer. <code>QueryMetric</code> và <code>QueryMetricsStore</code> lưu aggregated metrics theo SQL pattern. <code>DDLCollector/DDLContext</code> thu thập schema.
<code>com.dbquality.analysis</code>	Analysis Layer. <code>ScheduledAnalysisJob</code> là background thread chạy rule engine theo interval.
<code>com.dbquality.rule</code>	Rule Engine. Interface <code>Rule</code> và <code>MetricsBasedRule</code> , lớp <code>RuleEngine</code> , <code>Finding</code> , và 10 implementations.
<code>com.dbquality.explain</code>	Execution Plan Analysis. <code>ExplainCache</code> cache kết quả EXPLAIN theo pattern. Parsers cho MySQL, MariaDB, PostgreSQL, SQL Server.
<code>com.dbquality.report</code>	Output Layer. <code>ReportBuilder</code> tổng hợp report, <code>AnalysisResultStore</code> cache findings, <code>DashboardServer</code> phục vụ embedded HTTP dashboard.
<code>com.dbquality.ai</code>	AI Integration. Interface <code>LLMProvider</code> và 3 implementations: OpenAI, Claude, Gemini.
<code>com.dbquality.config</code>	<code>QualityConfig</code> chứa toàn bộ tham số, <code>QualityAutoConfiguration</code> cho Spring Boot zero-config.
<code>com.dbquality.util</code>	Tiện ích. <code>SQLFilter</code> , <code>SchemaFilter</code> , <code>SQLNormalizer</code> (chuẩn hóa SQL pattern).

2.1.3 Luồng dữ liệu

Luồng dữ liệu tổng thể đi qua 4 tầng:

1. **Interception Layer** — `QualityDataSource` bao bọc `DataSource` gốc. Mọi lời gọi `getConnection()` trả về `QualityConnection`, mọi `prepareStatement()` trả về `QualityPreparedStatement`.
2. **Collection Layer** — Khi SQL thực thi, `QualityPreparedStatement` đo thời gian, capture stack trace, normalize SQL qua `SQLNormalizer`, và update vào `QueryMetricsStore`. Không chạy rule ở đây.

3. **Analysis Layer** — `ScheduledAnalysisJob` chạy theo interval: warm-up `ExplainCache`, chạy 10 rules, lưu kết quả vào `AnalysisResultStore`.
4. **Output Layer** — Dashboard, JSON report đọc từ `AnalysisResultStore` (cache) cho findings, đọc trực tiếp từ `QueryMetricsStore` cho realtime metrics.

2.2 Cơ chế hoạt động

2.2.1 Thu thập metadata (DDL)

`DDLCollector` sử dụng `JDBC DatabaseMetaData` — API chuẩn, tương thích với mọi database có JDBC Driver. Quá trình thu thập gồm 4 bước lồng nhau:

1. `meta.getTables()` — liệt kê bảng người dùng, lọc system tables (Flyway, Liquibase, Spring Batch, Quartz) qua `SchemaFilter`
2. `meta.getColumns()` — thu thập tên cột, kiểu dữ liệu, nullable flag
3. `meta.getPrimaryKeys()` — xác định cột nào là Primary Key
4. `meta.getIndexInfo()` và `meta.getImportedKeys()` — thu thập index và foreign key

Kết quả lưu vào `DDLContext`.

2.2.2 Thu thập metrics (runtime)

Khi `executeQuery()` hoặc `executeUpdate()` được gọi:

1. **Lọc system SQL:** `SQLFilter` kiểm tra câu SQL có phải DML hợp lệ và không phải system query (HikariCP health check, Flyway migration, Hibernate schema validation).
2. **Đo thời gian:** `System.nanoTime()` trước và sau thực thi.
3. **Capture calledFrom:** Duyệt `Thread.currentThread().getStackTrace()`, lọc theo `INTERNAL_PREFIXES` và `INTERNAL_CONTAINS_PATTERNS`.
4. **Normalize SQL:** `SQLNormalizer` thay literal bằng ? để các lần thực thi cùng pattern được gom chung.
5. **Update metric:** Aggregate vào `QueryMetric` của pattern đó — tăng `callCount`, cập nhật min/max/avg duration, update `calledFrom` phổ biến nhất.

Việc lọc stack trace được thực hiện qua hai cơ chế song song:

(1) `INTERNAL_PREFIXES` — lọc theo `startsWith()`: JDK, Spring, Hibernate, Flyway, Jackson, Servlet containers, connection pools, JDBC drivers, và chính thư viện.

(2) `INTERNAL_CONTAINS_PATTERNS` — lọc theo `contains()`: Các runtime-generated proxy có tên dạng `io.demo.entity.User$HibernateProxy$xxo6c9Rz` — nếu chỉ dùng `startsWith()` sẽ lọt qua bộ lọc và `calledFrom` trở vào proxy thay vì code thật.

Listing 1: INTERNAL_CONTAINS_PATTERNS — lọc các runtime-generated proxy

```

1 public static final List<String> INTERNAL_CONTAINS_PATTERNS = List.of(
2     "$HibernateProxy$",      // Hibernate lazy loading proxy
3     "$$EnhancerBy",          // CGLIB / Spring AOP proxy
4     "$$_javassist_",         // Javassist proxy
5     "$Proxy",                 // JDK dynamic proxy
6     "$$Lambda$"              // Lambda generated classes
7 );

```

Kết hợp hai cơ chế đảm bảo `calledFrom` luôn trở vào method nghiệp vụ thật, ví dụ `EmployeeServiceImpl:133 -> toResponse()` thay vì `Employee$HibernateProxy$qPxTMV64.getFullName()`.

2.2.3 Phân tích Execution Plan với cache

Một lo ngại khi tiếp cận EXPLAIN-based analysis là *overhead có chấp nhận được không nếu chạy EXPLAIN cho mỗi query?* Câu trả lời đến từ một quan sát thực tế: ứng dụng production thường chỉ có **vài chục đến vài trăm unique SQL pattern**, dù tổng số lần thực thi có thể hàng triệu.

Từ đó, thư viện thiết kế `ExplainCache`:

- Mỗi unique SQL pattern (sau khi normalize) chỉ chạy EXPLAIN **1 lần**
- Kết quả được cache trong `ConcurrentHashMap`, thread-safe
- Lần thứ 2 trở đi: chỉ lookup map, không gọi DB

Với app có 100 unique patterns, tổng overhead của EXPLAIN chỉ là 100 query trong toàn bộ vòng đời app — gần như không đáng kể.

Hỗ trợ theo database vendor:

Bảng 2: Hỗ trợ EXPLAIN theo vendor

Database	Mô tả
MySQL / MariaDB	Parse output JSON, phát hiện: Full Table Scan (<code>access_type: ALL</code>), Full Index Scan (<code>access_type: index</code>), index khả dụng nhưng không được sử dụng.
PostgreSQL	Parse output JSON theo cấu trúc <code>Plan</code> node đệ quy, phát hiện: Seq Scan, Index Scan quét nhiều rows, Sort phải ghi xuống disk, Nested Loop sinh quá nhiều rows, planner estimate lệch xa thực tế.
SQL Server	Parse XML execution plan, phát hiện Table Scan và Index Scan.
Oracle	Nhận diện vendor; implementation EXPLAIN đầy đủ là hướng phát triển tiếp theo.

2.3 Mô tả Rule Engine

2.3.1 Cấu trúc Rule Engine

Rule Engine được thiết kế theo nguyên tắc Open/Closed: dễ mở rộng thêm rule mới mà không cần sửa code hiện có.

Có hai interface cho rule, tương ứng với hai loại nguồn dữ liệu:

Listing 2: Hai interface cho rule

```
1 // Rule dng DDL + raw SQL records
2 public interface Rule {
3     RuleResult analyze(DDLContext ddl, SQLContext sql);
4     String getName();
5     Severity getSeverity();
6 }
7
8 // Rule dng aggregated metrics
9 public interface MetricsBasedRule {
10     RuleResult analyze(DDLContext ddl, QueryMetricsStore metricsStore);
11     String getName();
12     Severity getSeverity();
13 }
```

`MetricsBasedRule` dành cho các rule dựa trên pattern (N+1, `SELECT *`, `FullTableScan`) — làm việc trên aggregated metrics thay vì raw records, tránh việc duyệt lại toàn bộ records mỗi lần phân tích.

2.3.2 Nhóm Rule dựa trên DDL

Các rule này phân tích `DDLContext` để phát hiện vấn đề cấu trúc database. Chúng kích hoạt ngay khi khởi động, không cần SQL traffic:

Bảng 3: Nhóm Rule phân tích Schema

Rule	Severity	Mô tả
MISSING_PRIMARY_KEY	CRITICAL	Bảng không có Primary Key. Nguy cơ trùng lặp dữ liệu, không tham chiếu được.
UNINDEXED_FOREIGN_KEY	HIGH	Cột FK không có index. JOIN sẽ full table scan.
SUSPICIOUS_DATA_TYPE	WARNING	FLOAT/DOUBLE cho tài chính (gây sai số làm tròn), VARCHAR cho ngày giờ (không sort/filter chính xác).

2.3.3 Nhóm Rule dựa trên Metrics

Các rule này dùng `QueryMetricsStore` làm input, làm việc trên aggregated metrics theo từng SQL pattern. Điểm nhấn là rule `N_PLUS_ONE` với phân loại thông minh dựa trên nhiều chỉ số.

Bảng 4: Nhóm Rule phân tích Metrics

Rule	Severity	Mô tả
SELECT_STAR	MEDIUM	Pattern dùng <code>SELECT *</code> . Mỗi unique pattern chỉ tạo 1 finding, kèm số lần xuất hiện và avg duration.
N_PLUS_ONE	Dynamic	Phân tích theo count, avg/min/max duration và variance. Severity HIGH nếu <code>callCount</code> ≥ 50 hoặc <code>total impact</code> $> 1000\text{ms}$; MEDIUM nếu ≥ 20 hoặc $> 200\text{ms}$.
SLOW_QUERY	HIGH	Pattern có <code>maxDurationMs</code> vượt <code>slowQueryThresholdMs</code> .

N+1 phân loại theo metrics: Đây là điểm khác biệt quan trọng so với các tool khác chỉ đếm số lần lặp. Thư viện phân tích đồng thời nhiều chỉ số:

- **Count cao + duration ổn định (variance thấp) + avg thấp** $\rightarrow$ N+1 điển hình của lazy loading. Message: “điển hình (lazy loading)”.
- **Count cao + avg cao ($\geq 50\text{ms}$)** $\rightarrow$ N+1 nghiêm trọng. Message: “nghiêm trọng”.
- **Variance cao ($\text{max/min} \geq 5$)** $\rightarrow$ có thể do cache miss/hit pattern. Message: “biến động”.

2.3.4 Nhóm Rule dựa trên EXPLAIN

Đây là nhóm rule **base hoàn toàn trên kết quả EXPLAIN thật** thay vì regex heuristic dựa trên SQL text. Cách tiếp cận này loại bỏ false positive: rule không báo nhầm khi nhìn thấy `LIKE '%...'` mà thực tế DB optimizer đã dùng full-text index.

Bảng 5: Nhóm Rule dựa trên EXPLAIN

Rule	Severity	Mô tả
FULL_TABLE_SCAN_CANDIDATE	Dynamic	Đọc kết quả EXPLAIN để phát hiện full scan. MySQL: <code>access_type=ALL</code> . PostgreSQL: <code>Seq Scan</code> . SQL Server: <code>Table Scan</code> . Severity tăng theo số rows được đọc và tần suất gọi.
MISSING_INDEX	MEDIUM	EXPLAIN xác định query không dùng index cho cột WHERE/JOIN. Resolve Hibernate alias (<code>e1_0</code>) về tên bảng thật.

Rule	Severity	Mô tả
UNUSED_INDEX	WARNING	Index có trong schema nhưng không xuất hiện trong bất kỳ EXPLAIN nào của session. Khuyến nghị verify trên production qua <code>sys.schema_unused_indexes</code> (MySQL) hoặc <code>pg_stat_user_indexes</code> (PostgreSQL) trước khi xóa.
NULLABLE_RISK	WARNING	Cột nullable xuất hiện trong WHERE clause của SQL runtime.

2.3.5 Tính minh bạch đầu vào

Một điểm quan trọng theo nguyên tắc thiết kế: **user phải verify được rule engine đang phân tích trên data gì**. Vì vậy dashboard cung cấp hai tab chuyên biệt:

- **Tab Collected Queries** — hiển thị toàn bộ SQL pattern đã thu thập: SQL text (normalized), số lần gọi, avg/min/max duration, vị trí code gọi nhiều nhất, thời gian thấy gần nhất.
- **Tab Schema Snapshot** — hiển thị DDL đã thu thập: danh sách tables, columns kèm kiểu dữ liệu và ràng buộc nullable, indexes, foreign keys.

User có thể đối chiếu finding với data thật: “rule báo `employees.email` thiếu index — mở Schema Snapshot xem có đúng không, mở Collected Queries xem cột này được dùng ở những câu nào, mỗi câu chạy bao nhiêu lần”. Điều này tăng credibility của tool.

2.3.6 Scoring

Điểm chất lượng tổng thể được tính từ 100, trừ dần theo số findings ở mỗi mức. Giới hạn tối đa mỗi mức tránh trường hợp nhiều findings cùng loại làm điểm âm:

Severity	Trừ / finding	Tối đa trừ	Ví dụ
CRITICAL	20 điểm	60 điểm	3+ findings: trừ hết 60
HIGH	10 điểm	30 điểm	3+ findings: trừ hết 30
MEDIUM	3 điểm	15 điểm	5+ findings: trừ hết 15
WARNING	1 điểm	5 điểm	5+ findings: trừ hết 5

2.4 Dashboard và Metrics API

2.4.1 Dashboard Realtime

Thư viện nhúng một HTTP server đơn giản sử dụng `com.sun.net.httpserver.HttpServer` có sẵn trong JDK — không cần thêm Tomcat, Netty hay bất kỳ server dependency nào. Dashboard phục vụ tại `http://localhost:9876` (port có thể cấu hình).

Dashboard gồm **6 tab**:

- **Overview** — score badge ở header (kèm tooltip công thức tính), nút Run Analysis Now, 4 metric cards, latency percentiles, top tables, top slow queries kèm nút EXPLAIN.
- **Findings** — danh sách findings phân trang, filter theo severity.
- **Collected Queries** — input transparency cho SQL pattern + metrics.
- **Schema Snapshot** — input transparency cho DDL.
- **AI** — AI-ready context và AI Insights từ LLM.
- **Project** — thông tin database, framework, JVM, memory.

2.4.2 Metrics API và các Endpoint

Listing 3: Ví dụ response từ GET /metrics

```

1 {
2   "totalSQLIntercepted": 859,
3   "uniquePatterns": 23,
4   "slowQueryCount": 0,
5   "nPlusOneDetected": 5,
6   "p50Latency": 1,
7   "p95Latency": 1,
8   "p99Latency": 3,
9   "errorRate": 0.0,
10  "score": 30
11 }
```

Các endpoint chính:

- GET /metrics — realtime metrics
- GET /collected-queries — toàn bộ SQL pattern + metrics
- GET /schema-snapshot — DDL đã thu thập
- GET /findings — findings từ lần analysis gần nhất (cache)
- GET /report — báo cáo đầy đủ
- GET /slow-queries — slow queries kèm EXPLAIN
- GET /analysis-status — trạng thái analysis job
- POST /analyze-now — trigger phân tích ngay

2.5 Hướng dẫn tích hợp

2.5.1 Yêu cầu môi trường

- Java 17 trở lên
- Maven 3.6+ hoặc Gradle 7+
- Mọi cơ sở dữ liệu quan hệ có hỗ trợ JDBC Driver

2.5.2 Thêm dependency

Listing 4: Maven dependency

```
1 <repositories>
2   <repository>
3     <id>jitpack.io</id>
4     <url>https://jitpack.io</url>
5   </repository>
6 </repositories>
7
8 <dependency>
9   <groupId>com.github.quanglam04</groupId>
10  <artifactId>database-quality-library</artifactId>
11  <version>master-SNAPSHOT</version>
12 </dependency>
```

2.5.3 Cấu hình

Listing 5: application.properties — cấu hình đầy đủ

```

1 # Enable/disable library (default: true)
2 quality.enabled=true
3
4 # Slow query threshold in milliseconds (default: 500)
5 quality.slow-query-threshold-ms=500
6
7 # N+1 detection threshold (default: 10)
8 quality.n-plus-one-threshold=10
9
10 # Analysis job
11 quality.analysis.scheduled=true           # Enable scheduled job
12 quality.analysis.interval=5m              # 30s/5m/1h/24h
13 quality.analysis.initial-delay=30s        # Wait before first run
14
15 # Embedded dashboard
16 quality.dashboard.enabled=true
17 quality.dashboard.port=9876
18
19 # Auto-export JSON on shutdown
20 quality.export.json.enabled=false
21 quality.export.json.path=quality-report.json
22
23 # AI integration (optional)
24 quality.ai.enabled=false
25 quality.ai.provider=gemini                 # openai / claude / gemini
26 quality.ai.api-key=YOUR_API_KEY
27 quality.ai.model=gemini-2.5-flash

```

2.5.4 Spring Boot zero-config

Listing 6: QualityAutoConfiguration.java

```

1 @AutoConfiguration(after = DataSourceAutoConfiguration.class)
2 @ConditionalOnClass(DataSource.class)
3 @ConditionalOnProperty(
4     prefix = "quality", name = "enabled",
5     havingValue = "true", matchIfMissing = true
6 )
7 public class QualityAutoConfiguration {
8
9     @Bean
10    @Primary
11    public DataSource qualityDataSource(DataSource dataSource) {
12        return new QualityDataSource(
13            dataSource, QualityConfig.fromClasspath());
14    }
15 }

```

Chỉ cần thêm dependency, không cần viết một dòng code nào.

3 Kết quả thực hiện và đánh giá

3.1 Môi trường thử nghiệm

Demo được thực hiện với 3 ứng dụng đại diện cho 3 framework và 3 database khác nhau:

Demo App	Framework	Database
HR Management	Spring Boot 3.3.0	MySQL 8.0.36
E-Commerce	Micronaut 4.x	PostgreSQL 16
Hospital Management	Plain Java + JDBC	SQL Server 2022

Tất cả đều dùng JDK 17, HikariCP, và được deploy qua Docker Compose.

3.2 Giá trị đem lại

3.2.1 Pain point 1: Production chậm nhưng không tái hiện được

Giải pháp: Tab Collected Queries cho thấy đầy đủ pattern + metrics. Tab Findings chỉ rõ FULL_TABLE_SCAN qua EXPLAIN thật.

Số liệu demo E-Commerce (PostgreSQL):

- Phát hiện 3 FULL_TABLE_SCAN qua EXPLAIN: products, orders, customers
- Tất cả đều do `findAll()` không có WHERE/index
- Mỗi finding kèm avg duration thực tế (1ms – 8ms)

Developer biết ngay câu nào full scan, ở đâu, mà không cần đợi sự cố production.

3.2.2 Pain point 2: Gặp N+1 nhưng không biết ở đâu

Giải pháp: Mỗi finding N+1 có trường `calledFrom` trỏ chính xác đến file:line trong code nghiệp vụ.

Số liệu demo Hospital (SQL Server) — 5 N+1 pattern được phát hiện:

Severity	Lặp	Tổng	calledFrom
HIGH	300 lần	432ms	WorkloadRunner:135 -> nPlusOneAppointmentsToPatient()
HIGH	300 lần	366ms	WorkloadRunner:158 -> nPlusOneAppointmentsToDoctor()
HIGH	200 lần	278ms	WorkloadRunner:179 -> nPlusOneRecordsToPrescriptions()
MEDIUM	20 lần	36ms	WorkloadRunner:237 -> filterAppointmentsByPatient()
WARNING	5 lần	17ms	WorkloadRunner:248 -> queryPrescriptionsByPatient()

Severity được phân loại dynamic dựa trên callCount và total impact — không chỉ đếm số lần lặp như các tool khác.

3.2.3 Pain point 3: Schema và code không đồng bộ

Giải pháp: Phân tích chéo — rule UNINDEXED_FK và UNUSED_INDEX chạy ngay khi startup, không cần chờ traffic.

Số liệu demo E-Commerce:

- 6 UNINDEXED_FK (HIGH) — mỗi cái kèm gợi ý cụ thể: `CREATE INDEX idx_order_items_order_id ON order_items(order_id)`
- 5 UNUSED_INDEX (WARNING) — kèm khuyến nghị verify qua `pg_stat_user_indexes` trước khi xóa

3.2.4 Pain point 4: Phát hiện vấn đề quá muộn

Giải pháp: DDL rules chạy ngay khi startup, phát hiện anti-pattern trước cả khi có traffic.

Số liệu demo Hospital:

- 1 MISSING_PRIMARY_KEY (CRITICAL) — bảng `prescriptions`
- 10 UNINDEXED_FK (HIGH)
- 8 SUSPICIOUS_DATA_TYPE (WARNING) — VARCHAR cho date, FLOAT cho salary và cost

Toàn bộ phát hiện được ngay khi app khởi động lần đầu — chưa có request nào chạy qua.

3.2.5 Pain point 5: Nhiều tool rời rạc

Giải pháp: Dashboard 6 tab tổng hợp đầy đủ — collected queries, schema, findings, EXPLAIN, project info, AI insights — trong một URL duy nhất.

3.3 Kết quả tổng hợp

Metric	HR/MySQL	E-com/PG	Hospital/SQLS
Overall Score	62	30	30
Total SQL intercepted	338	748	859
Unique patterns	24	14	23
N+1 detected	4	4	5
DDL findings	14	14	19
SQL findings	9	10	16
P99 latency	6ms	2ms	3ms

3.4 Đánh giá overhead khi tích hợp thư viện

Một câu hỏi quan trọng khi đưa thư viện vào production: **thư viện có làm chậm ứng dụng không, và chậm bao nhiêu?** Phần này phân tích overhead theo từng layer dựa trên thiết kế, kèm số liệu đo thực tế bằng Apache Bench trên demo HR Management.

3.4.1 Overhead trên hot path (mỗi câu SQL)

Hot path là đoạn code chạy mỗi khi ứng dụng thực thi SQL. Thư viện thiết kế Collection Layer cực nhẹ với các thao tác sau cho mỗi câu SQL:

1. Đo thời gian bằng `System.nanoTime()` trước và sau `execute()` — chi phí < 100 nanosecond.
2. Lọc system SQL qua `SQLFilter` (regex check) — chi phí ~ 1 microsecond.
3. Capture stack trace qua `Thread.currentThread().getStackTrace()` — đây là thao tác nặng nhất, khoảng 10–50 microsecond tùy độ sâu stack.
4. Lọc stack trace để tìm `calledFrom` qua hai bộ lọc `INTERNAL_PREFIXES` và `INTERNAL_CONTAINS_PATTERNS` — chi phí ~ 5 microsecond.
5. Normalize SQL pattern qua `SQLNormalizer` — chi phí ~ 10 microsecond cho câu SQL trung bình.
6. Update vào `QueryMetricsStore` (`ConcurrentHashMap.compute()`) — chi phí ~ 1 microsecond.

Tổng overhead lý thuyết mỗi câu SQL: khoảng 20–100 microsecond (0.02–0.1ms), trong khi một câu SQL thông thường chạy 1–100ms. Tỷ lệ overhead so với latency thực của SQL thường $< 1\%$.

Điều quan trọng: Collection Layer **không chạy rule engine, không gọi EXPLAIN, không phân tích DDL** ở hot path. Toàn bộ tác vụ nặng được dời sang Analysis Layer.

3.4.2 Overhead trên Analysis Layer (background)

Analysis Layer chạy trên background thread riêng (`ScheduledAnalysisJob`) theo interval cấu hình được (mặc định 5 phút). Mỗi lần chạy thực hiện các tác vụ:

- **Warm-up ExplainCache:** chạy EXPLAIN cho các unique pattern *chưa có trong cache*. Lần đầu sau khi app khởi động sẽ chạy EXPLAIN cho toàn bộ patterns đã thấy. Các lần sau chỉ chạy cho pattern mới — thường rất ít.
- **Chạy 10 rules:** duyệt DDL context và metrics store. Chi phí $O(\text{tables} + \text{patterns})$.
- **Lưu kết quả vào AnalysisResultStore:** thao tác đơn giản, không đáng kể.

Background thread này **không nằm trên đường đi của request user**, nên dù tốn vài chục đến vài trăm millisecond mỗi lần chạy, không ảnh hưởng đến latency thực tế mà user cảm nhận. Ngay cả khi user mở dashboard, API endpoint cũng chỉ **đọc cache** chứ không trigger phân tích.

3.4.3 Overhead bộ nhớ

Thư viện sử dụng các cấu trúc dữ liệu in-memory để lưu trữ metrics và DDL snapshot:

- **QueryMetricsStore:** `ConcurrentHashMap<String, QueryMetric>` với mỗi entry $\sim 200\text{--}500$ bytes. Với 100 unique patterns, tốn $\sim 50\text{KB}$.

- **DDLContext**: lưu danh sách tables, columns, indexes, FKs. Với database 50 bảng và 500 cột, tốn $\sim 200\text{KB}$.
- **ExplainCache**: `ConcurrentHashMap<String, ExplainResult>` với mỗi entry $\sim 1\text{--}5\text{KB}$ (JSON/XML plan). Với 100 patterns, tốn $\sim 500\text{KB}$.
- **AnalysisResultStore**: cache findings từ lần phân tích gần nhất, thường $< 100\text{KB}$.

Tổng dung lượng bộ nhớ thư viện sử dụng: thường $< 5\text{MB}$ ngay cả với app production có hàng trăm pattern và database lớn. So với heap JVM Spring Boot điển hình (256MB–2GB), tỷ lệ $< 2\%$.

Thư viện không tích lũy unbounded: số lượng entries bị giới hạn bởi số unique pattern (thường vài chục đến vài trăm), không bởi số lần thực thi.

3.4.4 Đo overhead thực tế bằng Apache Bench

Để đánh giá overhead thực tế một cách định lượng, demo app HR Management được benchmark bằng Apache Bench với 2 chế độ:

- **Baseline**: dùng `HikariDataSource` trực tiếp, `quality.enabled=false`.
- **Có thư viện**: dùng `QualityDataSource` bao bọc `HikariCP`, `quality.enabled=true`.

Methodology:

- Workload: `ab -n 1000 -c 10` (1000 requests, concurrency 10).
- Mỗi mode chạy **3 lần liên tiếp**; loại bỏ run đầu tiên (cold start do JIT compilation, HikariCP pool warm-up, MySQL buffer pool ấm dần).
- Lấy trung bình hai run sau làm số liệu công bố.
- Môi trường: cùng Docker Compose stack, MySQL 8.0, mỗi mode khởi động lại container để clear buffer pool, có warm-up 500 request trước khi đo.

Kết quả endpoint GET /api/employees (response 4368 bytes, có JOIN giữa nhiều bảng):

Chỉ số	Baseline	Có thư viện	Chênh lệch
P50 latency	43.5 ms	44.0 ms	+0.5 ms
P95 latency	78.5 ms	74.0 ms	−4.5 ms
P99 latency	104.5 ms	89.0 ms	−15.5 ms
Mean latency	46.5 ms	47.0 ms	+0.5 ms
Throughput	211.3 req/s	209.4 req/s	−0.9%

Kết quả endpoint GET /api/departments (response 786 bytes):

Chỉ số	Baseline	Có thư viện	Chênh lệch
P50 latency	66.0 ms	65.5 ms	-0.5 ms
P95 latency	103.0 ms	100.0 ms	-3.0 ms
P99 latency	127.0 ms	123.0 ms	-4.0 ms
Mean latency	69.0 ms	68.0 ms	-1.0 ms
Throughput	142.6 req/s	144.8 req/s	+1.5%

Phân tích kết quả: Mean latency giữa hai chế độ chênh lệch trong khoảng $\pm 1\text{ms}$ (từ -1.0ms đến $+0.5\text{ms}$), throughput dao động $\pm 1.5\%$. Trong cả hai endpoint, sự khác biệt giữa baseline và with-lib **nhỏ hơn variance tự nhiên của benchmark** (standard deviation $\sim 10\text{--}15\text{ms}$ cho P99 trên local Docker với concurrency 10).

Đáng chú ý là endpoint **departments** cho thấy with-lib có mean latency thấp hơn baseline 1ms — điều này không phải vì thư viện làm app nhanh hơn (vô lý về mặt logic), mà do biến động ngẫu nhiên giữa các lần đo: MySQL InnoDB Buffer Pool có trạng thái khác nhau, OS Page Cache, JIT compilation timing, và các process nền của hệ điều hành.

Kết luận: Overhead thực tế của thư viện **nhỏ hơn noise level của phương pháp đo** — không phát hiện được sự khác biệt có ý nghĩa thống kê giữa baseline và with-lib. Kết hợp với phân tích lý thuyết (20–100 microsecond mỗi query), có thể kết luận overhead nằm dưới ngưỡng **1ms mỗi request**, hoàn toàn chấp nhận được cho dev/staging và phần lớn use case production.

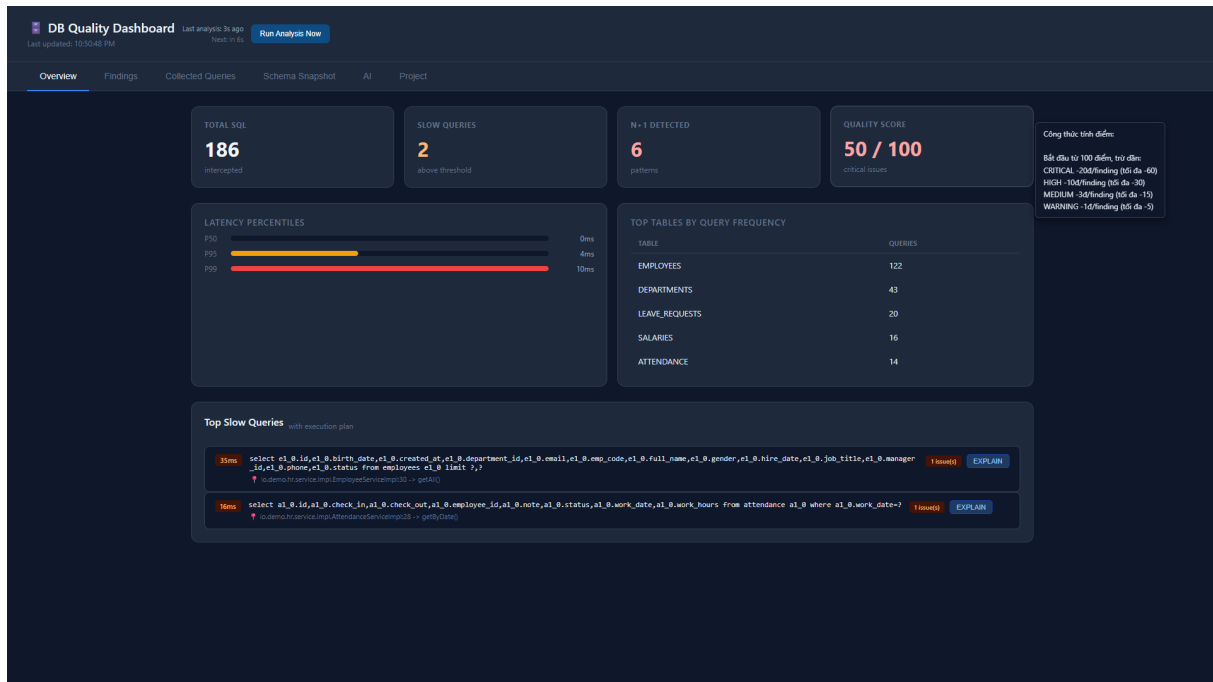
3.4.5 Khuyến nghị triển khai

Dựa trên đánh giá trên, thư viện được khuyến nghị dùng trong các môi trường:

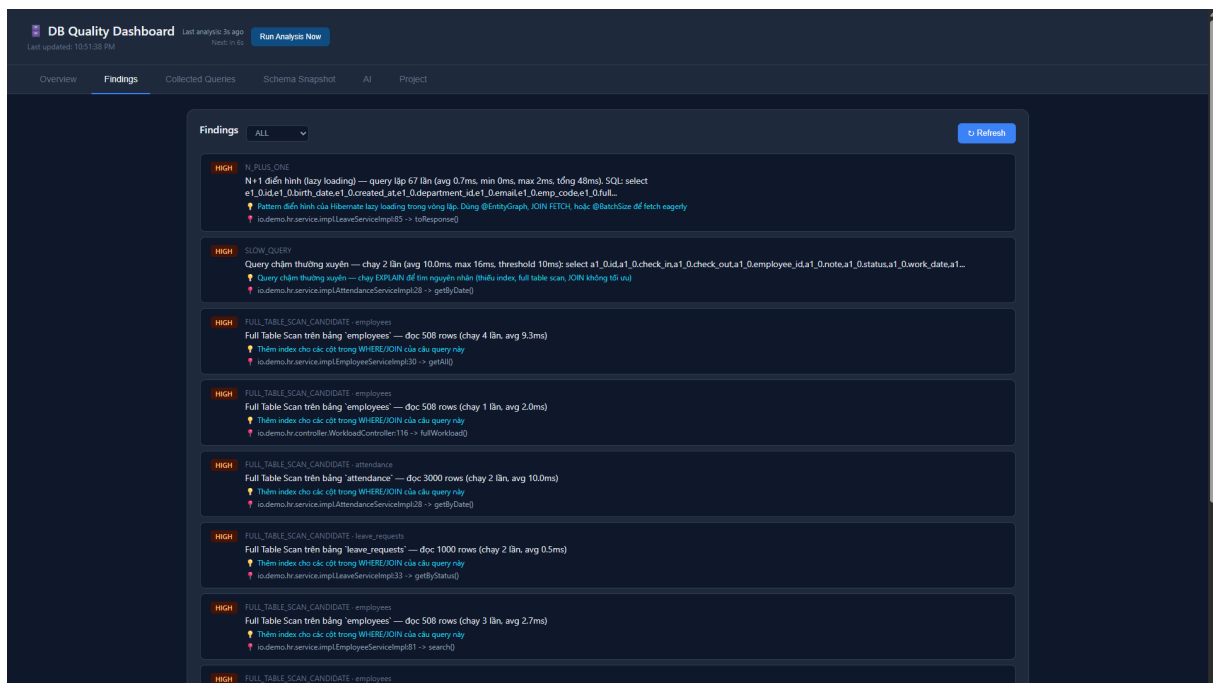
- **Development:** bật toàn bộ tính năng, interval analysis ngắn (30s–5m) để feedback nhanh.
- **Staging / UAT:** bật toàn bộ, interval analysis dài hơn (1h–24h), dùng để phát hiện vấn đề trước khi đẩy lên production.
- **Production:** có thể bật với interval analysis 24h hoặc tắt analysis, chỉ giữ collection để thu thập metrics. Trong trường hợp cực kỳ nhạy cảm về latency, có thể tắt hoàn toàn qua `quality.enabled=false`.

Thư viện không thiết kế để thay thế APM cho production — mục tiêu chính là phát hiện vấn đề chất lượng database **trước khi** đến với end-user, ở giai đoạn dev và staging.

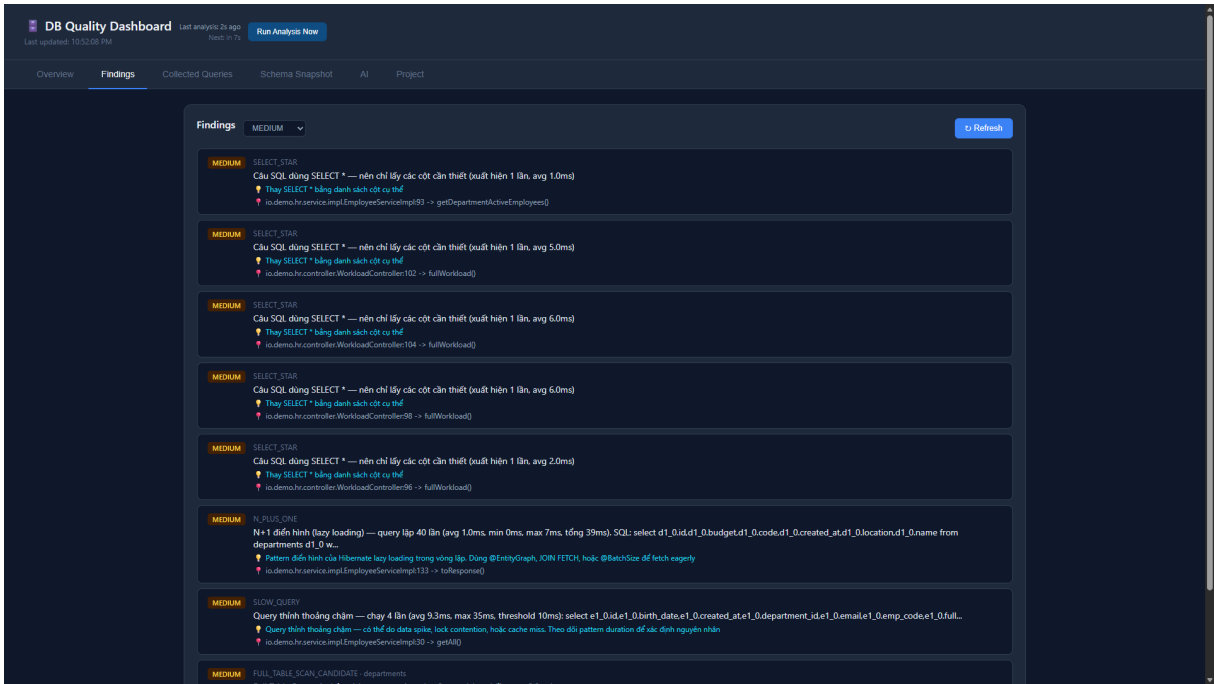
3.5 Dashboard demo



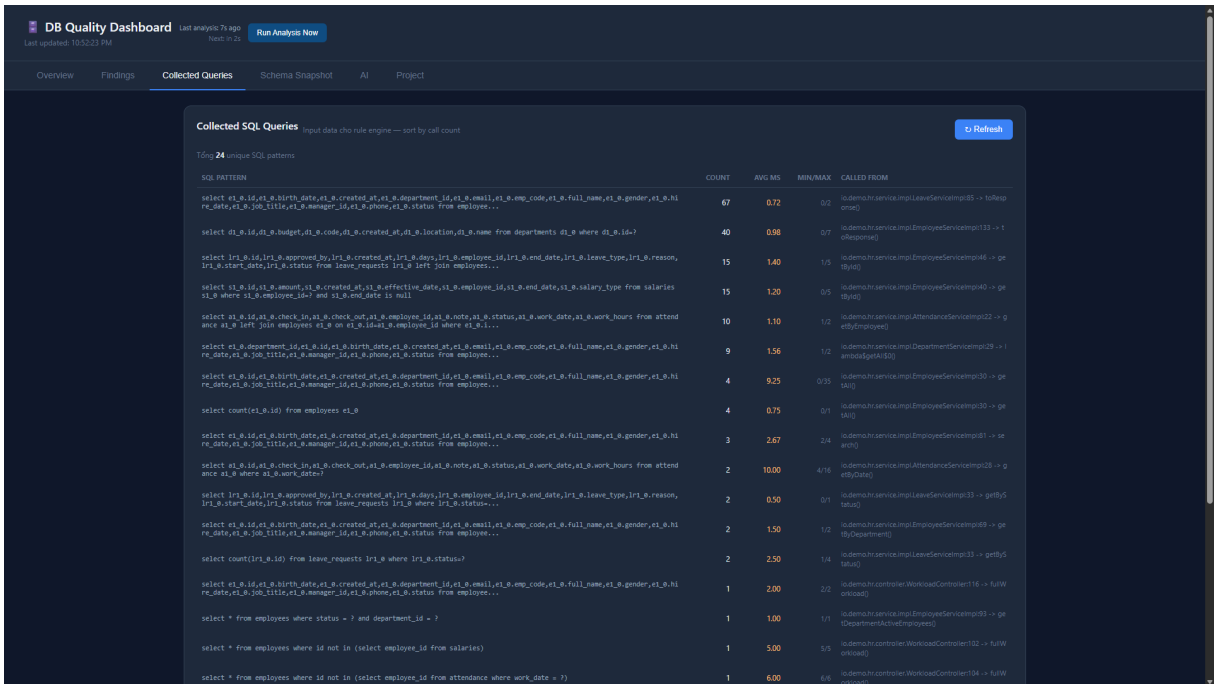
Hình 1: Tab Overview — score badge ở header (kèm tooltip công thức tính), nút Run Analysis Now, 4 metric cards, latency percentiles, top tables và top slow queries kèm nút EXPLAIN



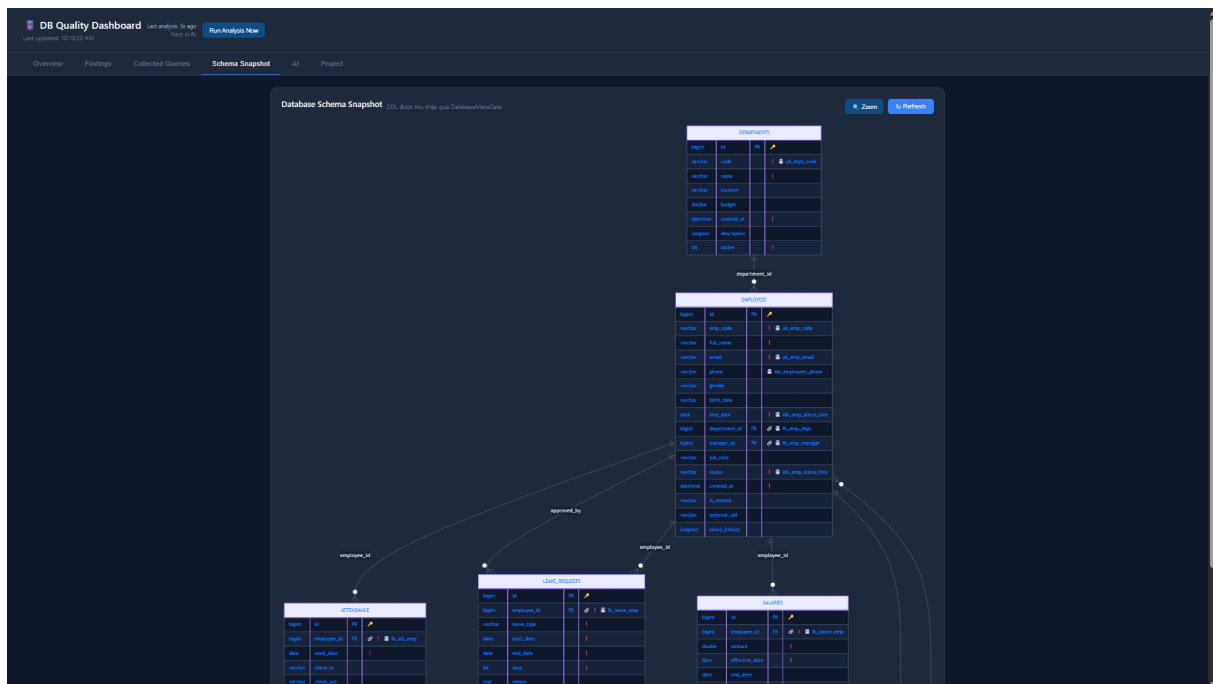
Hình 2: Tab Findings — danh sách findings sắp xếp theo severity, mỗi finding có rule name, severity badge, message với metrics thực tế, recommendation và vị trí calledFrom trong code nghiệp vụ



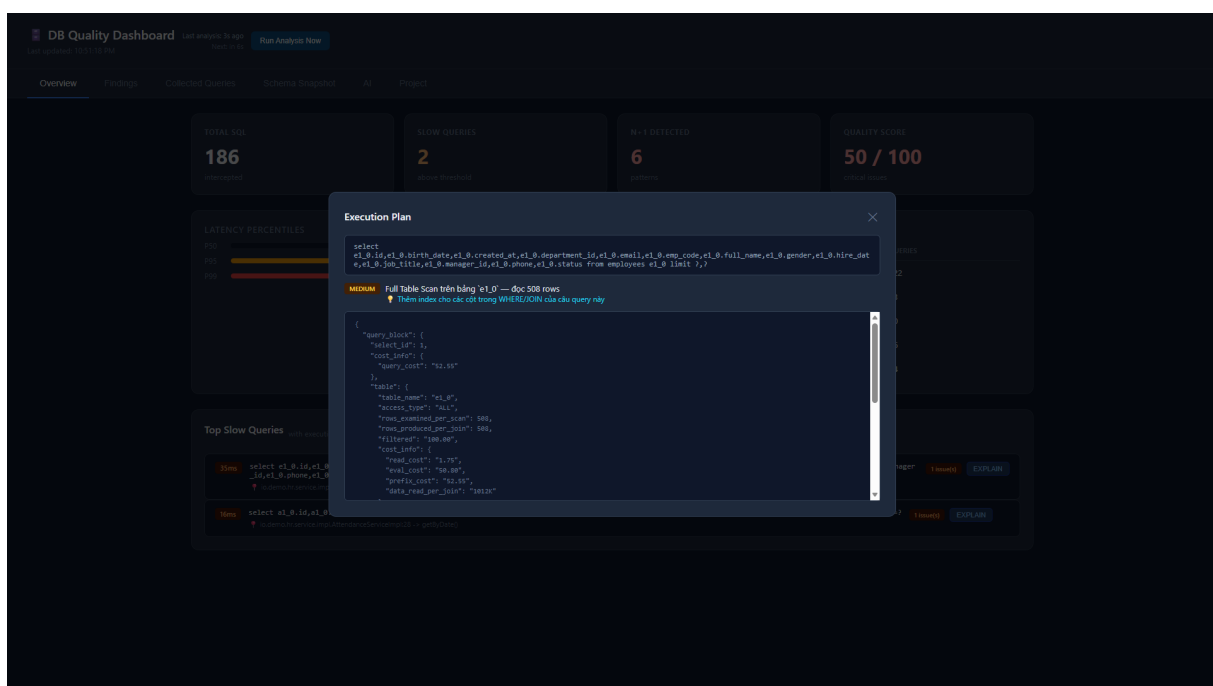
Hình 3: Tab Findings — filter theo severity để tập trung xử lý vấn đề ưu tiên cao trước



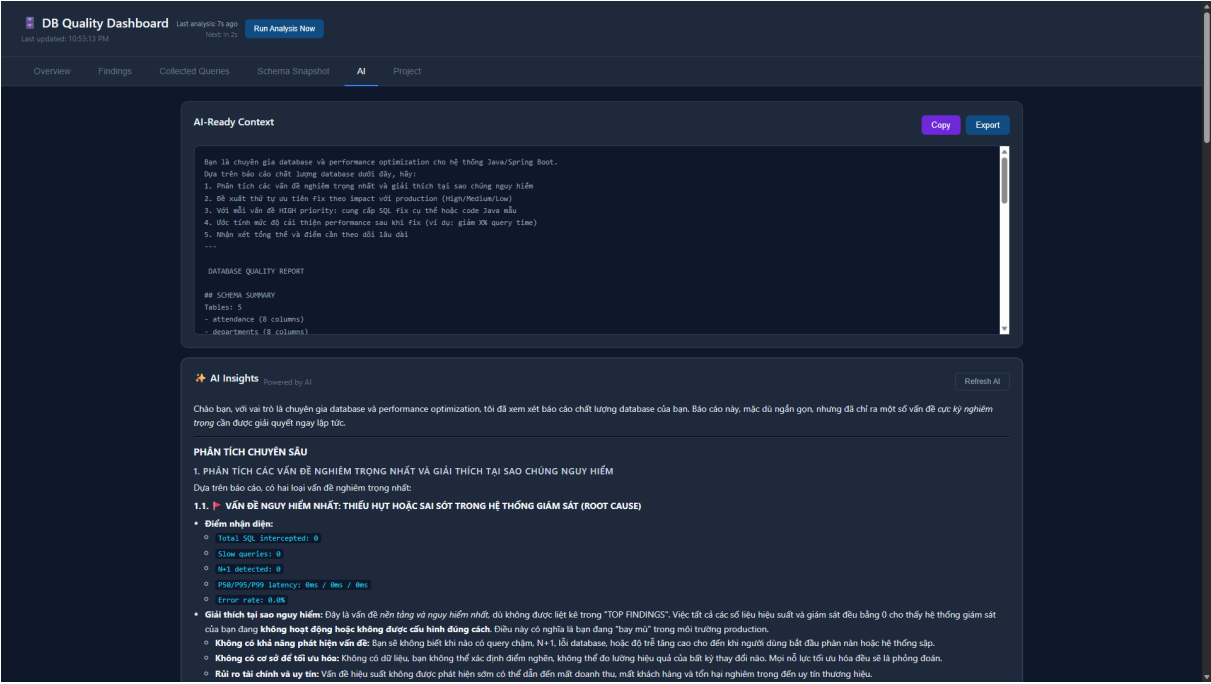
Hình 4: Tab Collected Queries — input transparency của rule engine. Hiển thị toàn bộ SQL pattern đã được normalize, kèm call count, avg/min/max duration và vị trí code gọi nhiều nhất



Hình 5: Tab Schema Snapshot — input transparency của rule engine cho DDL. Hiển thị cấu trúc schema thu thập qua DatabaseMetaData: tables, columns kèm kiểu dữ liệu và ràng buộc nullable, indexes, foreign keys



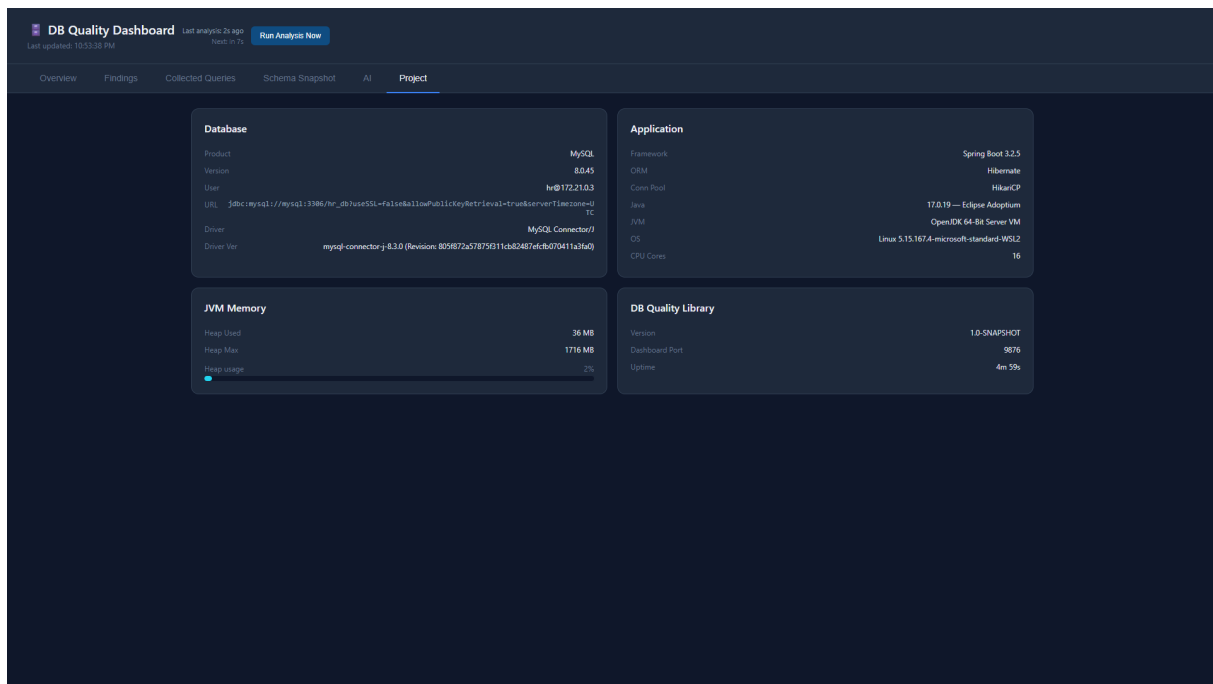
Hình 6: Modal Execution Plan — hiển thị khi bấm nút EXPLAIN trên slow query. Bao gồm findings phát hiện được từ plan (full scan, low selectivity, sort to disk) và raw JSON/XML output của EXPLAIN



Hình 7: Tab AI (phần trên) — AI-ready context: prompt structured được build từ schema summary, top findings, metrics, slow queries. Có thể copy hoặc export để dùng với bất kỳ LLM nào



Hình 8: Tab AI (phần dưới) — AI Insights được render từ markdown response của LLM, hỗ trợ code block, table, list. LLM phân tích sâu hơn và đề xuất SQL fix cụ thể



Hình 9: Tab Project — thông tin runtime: database product/version, framework, ORM, connection pool, JVM heap usage với progress bar, và uptime

3.6 Đánh giá tổng thể

- **Minh bạch đầu vào:** Tab Collected Queries và Schema Snapshot cho phép user verify rule engine đang phân tích trên data gì — tăng credibility của findings.
- **EXPLAIN-based phát hiện chính xác:** Các rule FullTableScan, MissingIndex, UnusedIndex base trên kết quả EXPLAIN thật, không có false positive. N+1 phân loại theo metrics (count + variance + duration), không chỉ đếm số lần lặp.
- **Tách rõ runtime collection và scheduled analysis:** Interval analysis cấu hình được linh hoạt qua `application.properties`.
- **Độ chính xác:** Thư viện phát hiện đúng các vấn đề thực tế trên cả 3 demo app. Không có false positive trong các test case với 34 test pass.
- **Tính chính xác của calledFrom:** Trỏ đúng đến file:line nghiệp vụ — nhờ kết hợp 2 cơ chế lọc stack trace (`startsWith` và `contains` cho proxy class).
- **Overhead thực tế:** Đo bằng Apache Bench cho thấy chênh lệch latency giữa baseline và with-lib nằm trong noise level của benchmark, dưới 1ms mỗi request — xác nhận thiết kế Collection Layer cực nhẹ đạt mục tiêu đặt ra.
- **Zero-config với Spring Boot:** Chỉ cần thêm dependency, không cần annotation hay config class.
- **Tính tương thích:** Hoạt động đúng trên cả 3 stack hoàn toàn khác nhau (Spring Boot/MySQL, Micronaut/PostgreSQL, Plain Java/SQL Server).
- **AI Integration tùy chọn:** Khi không bật, tool vẫn hoạt động đầy đủ với rule-based output.

4 Kết luận

4.1 Tóm tắt kết quả

Đề tài đã xây dựng thành công thư viện **Database Quality Library** với cách tiếp cận khác biệt: thay vì là một tool monitoring tổng quát, thư viện tập trung giải quyết năm pain point cụ thể của developer thông qua bốn nguyên tắc thiết kế.

Các tính năng đã hoàn thành:

- **JDBC Interception** bằng Decorator Pattern, hoàn toàn trong suốt với code nghiệp vụ.
- **Tách Collection và Analysis Layer** — collection ghi metric realtime, analysis chạy theo interval cấu hình được.
- **Rule Engine 10 rules** — DDL-based, Metrics-based, EXPLAIN-based. N+1 phân loại thông minh theo count + variance + duration.
- **Input transparency** — Collected Queries + Schema Snapshot cho phép user verify rule engine đang phân tích trên data gì.
- **ExplainCache** — mỗi pattern chỉ chạy EXPLAIN 1 lần, overhead minimal.
- **Dashboard 6 tab** không cần server dependency.
- **AI Integration** optional với 3 providers.
- **Spring Boot zero-config** qua autoconfiguration.
- **Verified trên 3 stack:** Spring Boot/MySQL, Micronaut/PostgreSQL, Plain Java/SQL Server.
- **Overhead thực tế dưới noise level** — xác nhận qua Apache Bench benchmark.

4.2 Hướng phát triển tương lai

- Triển khai đầy đủ EXPLAIN parser cho Oracle.
- Hỗ trợ persistent storage để lưu lịch sử phân tích, so sánh giữa các phiên bản deploy.
- Thêm alerting webhook (Slack, email) khi score giảm sâu hoặc xuất hiện CRITICAL finding mới.
- Hoàn thiện DeepSeekProvider và GrokProvider.
- Hỗ trợ Gradle plugin để tự động phân tích trong CI/CD pipeline.
- Custom rule registration — cho phép user đăng ký rule riêng qua SPI.

Tài liệu tham khảo

1. Spring Framework Team. *Spring Boot Auto-configuration*. <https://docs.spring.io/spring-boot/docs/current/reference/html/auto-configuration-classes.html>
2. MySQL Documentation. *EXPLAIN Output Format*. <https://dev.mysql.com/doc/refman/8.0/en/explain-output.html>
3. PostgreSQL Documentation. *Using EXPLAIN*. <https://www.postgresql.org/docs/current/using-explain.html>
4. Microsoft Documentation. *Display Execution Plans (SQL Server)*. <https://learn.microsoft.com/en-us/sql/relational-databases/performance/display-execution-plans>
5. Tenório. *Understanding the N+1 Problem in Databases*. LinkedIn, 2024. <https://www.linkedin.com/pulse/understanding-n1-problem-databases-common-pitfall-software-tenorio-r4q5f/>
6. Oracle Corporation. *Java Platform SE 17 — Interface DatabaseMetaData*. <https://docs.oracle.com/en/java/javase/17/docs/api/java.sql/java/sql/DatabaseMetaData.html>
7. Apache Software Foundation. *ab — Apache HTTP server benchmarking tool*. <https://httpd.apache.org/docs/current/programs/ab.html>